

>saturnCFD

# saturnCFD v1

## MPI and GPU finite-volume CFD with validated methodology

Ananth Narayan

Indian Institute of Technology Madras

31 October 2026

# Outline


1. Motivation and scope
2. What is implemented
3. Mesh and configuration
4. Parallel model
5. Finite-volume method
6. Verification and validation
7. Commands and tests
8. Status and roadmap

# What saturnCFD is

## Goal

A single finite-volume code for scalar transport and incompressible flow, portable across CPU and GPU, parallel through MPI, driven entirely by YAML, and verified before it is trusted.

- Structured hexahedral mesh imported from Gmsh `.msh`.
- 3D from the start; a one-cell-thick mesh is treated as 2D.
- Solvers: Burgers, heat/convection–diffusion, Stokes, incompressible Navier–Stokes.
- Kokkos for CPU/GPU; MPI for distributed memory.
- Every numerical choice is documented, cited and tested.

v0 is preserved on `v0-archive`. v1 is a deliberate rewrite, not a patch on the old AMR container.

# Why this order?

1. Burgers: nonlinear convection, shocks, limiters, diffusion.
2. Heat: scalar transport, Péclet effects, boundary layers.
3. Stokes: vector elliptic operator, pressure coupling, linear solver.
4. Incompressible Navier–Stokes: all of the above together.

**Idea**

Each PDE tests one more ingredient. A failure is localised before the next solver is added.

Compressible flow, AMR and two-phase flow are later milestones.

# Implemented substrate

- Gmsh 2.2 reader; structured-hex classification and `checkMesh` report.
- Default `.vtu` + `.json`; optional `-writeVTK` patches and `.pvd`.
- yaml-cpp configuration with a typed schema and generated templates.
- SI-only validation: non-SI input is rejected, not converted.
- Single-dash CLI with help topics and shell completion.
- MPI Cartesian partition and 26-neighbour halo exchange.
- Kokkos `CellField`/`FaceField`, device pack/unpack halo, structured `LocalMesh`.
- Interpolation, Gauss gradient, divergence, non-orthogonal Laplacian and Dirichlet/Neumann ghosts.
- Upwind/Rusanov/MUSCL convection, TVD limiters and advection–diffusion MMS.
- Divergence theorem on a sheared mesh; simplified Rhie–Chow flux.
- PETSc 3.19.6 through `saturnLinAlg`; 1D/3D Poisson and all-Neumann nullspace tests.
- Solver interface, registry, `TimeControl` and explicit Simulation loop (forward Euler, SSPRK2/3).
- 1D/2D/3D `saturnBurger`: MUSCL/Rusanov flux, diffusion, Dirichlet and periodic boundaries.
- 1D/2D/3D `saturnHeat`: prescribed velocity, convection–diffusion, periodic Fourier validation.
- `output.vtkEvery`: `.vtu` frames plus a ParaView `.pvd` collection.
- Periodic Laplacian, gradient and divergence MMS at order 2 on 1/2/4 ranks.
- 46 CTest cases, regression tests, Doxygen HTML/PDF.

# Architecture

| Layer | Responsibility |
|---|---|
| `core` | types, SI units, errors, timer |
| `config` | yaml-cpp, schema, validation, templates |
| `mesh` | Gmsh, `checkMesh`, structured view, VTK |
| `comm` | partition, ghost ownership, halo exchange |
| `fv` | fields, gradient, divergence, Laplacian, fluxes |
| `linalg` | MPI PETSc systems |
| `simulation` | solver registry, time loop, diagnostics |
| `solvers` | Burgers, heat, Stokes, incompressible |

A solver owns its equations. It does not own the mesh, the halo, the YAML parser or the linear algebra.

# Structured mesh and `checkMesh`

- Gmsh 2.2 ASCII: points, hexahedra, boundary physical groups.
- Classifier walks the cell dual graph and assigns a contiguous $(i, j, k)$.
- Rejects tetrahedra, non-manifold faces, negative volumes and non-contiguous hex meshes.
- Reports counts, patches, volume/area ranges, non-orthogonality, skewness, aspect ratio, structured flag and dimension.

**2D rule**

A one-cell-thick 3D mesh with `empty` patches is 2D. One internal representation, one solver path.

Gmsh [5]; OpenFOAM's `checkMesh` convention is a methodology reference.

# YAML and SI units

```
solver: common
units:
  system: SI
mesh:
  file: channel.msh
time:
  end: 1.0
  cfl: 0.4
```

- yaml-cpp parses; the schema validates.
- Unknown keys are errors.
- `-dumpConfig` generates the template from the same registration the solver reads.
- `-checkConfig` runs all validation before allocation.

## SI only in v1

`units.system: SI` or no block is accepted. `CGS`, `length: mm`, etc. are hard errors. A future units layer converts at the configuration boundary, never in a kernel.

# MPI decomposition

| | |
|---|---|
| global grid | $n_x \times n_y \times n_z$ |
| process grid | $p_x \times p_y \times p_z$ |
| local block | $[i_0, i_1] \times [j_0, j_1] \times [k_0, k_1]$ |

- `MPI_Dims_create` chooses the process grid; `MPI_Cart_create` handles periodicity.
- 2D uses $p_z = 1$; 1D uses $p_y = p_z = 1$.
- Remainders are distributed one cell at a time from the low index.
- No METIS: a structured mesh does not need a graph partition.

## Why Cartesian blocks?

Deterministic neighbours, compact messages and a direct cell $\leftrightarrow$ rank mapping.

# No shared cells: owner and ghost

- Every global cell has **one owner rank**.
- Only the owner writes the valid value.
- The neighbour keeps a **read-only ghost copy**.
- Two ghost layers support MUSCL/TVD.
- 26-neighbour exchange: faces, edges, corners.

**Shared face, not shared cell**

Both ranks compute the same face flux from the same valid/ghost pair. Each rank updates only its own cell. No double counting.

Halo order: local periodic copies → packed MPI → physical boundary conditions.

# CPU and GPU execution

- Kokkos selects the backend at **build** time: Serial/OpenMP or CUDA/HIP.
- CPU and GPU are separate build directories: `buildCPU` and `buildGPU`.
- `mpirun` launches MPI ranks; the compiled backend decides whether a rank uses a CPU or a GPU.
- Performance model: **one rank per GPU**.
- One GPU: `mpirun -np 1`. Four GPUs: `mpirun -np 4`.
- Halo first implementation: device pack $\rightarrow$ host MPI $\rightarrow$ device unpack.

## Case files are hardware-independent

CPU/GPU is not in `case.yaml`. Physics stays portable.

# Collocated cell-centred FVM

- Velocity and pressure at cell centroids.
- Face fluxes reconstructed from the two adjacent cells.
- Rhie–Chow correction suppresses checkerboard pressure.
- Coupled saddle-point [1, 4] for steady Stokes.
- SIMPLE/SIMPLEC remain optional segregated paths for Navier–Stokes; PISO for transient flow.

## Why not MAC?

Staggered MAC gives exact discrete continuity, but it splits the velocity components onto different meshes and fights the vector form, `.msh` import and the 2D/3D rule. Collocated FV matches OpenFOAM and the default Basilisk/Nektar++ approach [14, 2]; Rhie–Chow [10] restores pressure–velocity stability.

# Governing equations

$$\text{Burgers:} \quad \frac{\partial u}{\partial t} + \nabla \cdot (u\mathbf{a}) = \nu \nabla^2 u + s$$

$$\text{Heat:} \quad \frac{\partial T}{\partial t} + \nabla \cdot (\mathbf{u} T) = \nabla \cdot (\alpha \nabla T) + S$$

$$\text{Stokes:} \quad -\nabla p + \mu \nabla^2 \mathbf{u} = \mathbf{f}, \qquad \nabla \cdot \mathbf{u} = 0$$

$$\text{Incompressible NS:} \quad \frac{\partial \mathbf{u}}{\partial t} + \nabla \cdot (\mathbf{u} \otimes \mathbf{u}) = -\frac{1}{\rho} \nabla p + \nu \nabla^2 \mathbf{u} + \frac{\mathbf{f}}{\rho},$$
$$\nabla \cdot \mathbf{u} = 0$$

Projection/pressure correction from Chorin [3]; SIMPLE/PISO from Patankar [9] and Issa [7]; Rhie–Chow [10]; coupled saddle point from Benzi et al. [1] and Elman et al. [4].

# Vector form and linear algebra

## Discrete momentum

$$\mathbf{M}\frac{d\mathbf{u}}{dt} + \mathbf{C}(\mathbf{u})\mathbf{u} = -\mathbf{G}p + \mathbf{D}\mathbf{u} + \mathbf{f}$$

- One vector unknown per cell: $u, v, w$.
- One sparsity pattern; block size 3 or three right-hand sides.
- PETSc: `Mat`, `Vec`, `KSP`, `PC`.
- Pressure Poisson: CG + GAMG/HYPRE; Neumann null space handled explicitly.
- Coupled Stokes/NS: `MPIAIJ` with $(u, v, w, p)$ per cell, GMRES/FGMRES + `PCFIELDSPLIT` Schur preconditioner.
- Velocity block GAMG/ILU; pressure block GAMG/HYPRE; nullspace attached with `MatNullSpace`.
- No home-grown Krylov solver.

# Stokes: coupled saddle point

$$\begin{bmatrix} A & B^T \\ B & 0 \end{bmatrix} \begin{bmatrix} \mathbf{u} \\ p \end{bmatrix} = \begin{bmatrix} \mathbf{f} \\ 0 \end{bmatrix}$$

- One PETSc solve for velocity and pressure; no SIMPLE outer iteration for Stokes.
- Collocated FV + Rhie–Chow [10] stabilisation.
- Schur factorisation $S = -BA^{-1}B^T$; approximate $A^{-1}$ and $S^{-1}$ with AMG/ILU; GMRES handles the coupling [1, 4].
- Pressure nullspace attached for all-Neumann cases; mean pressure reported separately.
- The same block is reused inside Navier–Stokes Picard/Newton or time-step iterations.

## Why coupled?

Stokes is linear and steady; pressure is a Lagrange multiplier. A coupled solve avoids under-relaxation and pseudo-time tuning.

# Stokes validation plan

| Case | Quantity | Reference |
|---|---|---|
| Poiseuille 2D/3D | velocity profile | analytic parabola |
| Couette 2D/3D | velocity profile | analytic line |
| MMS 2D/3D | velocity/pressure order | manufactured |
| Exit boundary | 3D outflow | Sankhla & Das [13] |
| Nullspace | mean pressure | constant mode |

## Status

Poiseuille and Couette pass in 2D/3D; 2D runs on 1/2/4 ranks. The 3D multi-rank DMDA assembly, MMS and exit-boundary cases follow.

# Stokes: first validated results

| Case | mesh | $L^2$ | note |
|---|---|---|---|
| Poiseuille 2D | $16^2$ | $4.88 \times 10^{-4}$ | second order |
| Poiseuille 2D | $32^2$ | $1.22 \times 10^{-4}$ | second order |
| Poiseuille 2D | $64^2$ | $3.05 \times 10^{-5}$ | example |
| Couette 2D | $64^2$ | $7.5 \times 10^{-13}$ | exact line |
| Poiseuille 3D | $24^2 \times 8$ | $2.17 \times 10^{-4}$ | extruded |
| Couette 3D | $24^2 \times 8$ | $5.7 \times 10^{-12}$ | exact line |

## Solver

Coupled velocity-pressure saddle point, PETSc direct LU for this validation slice; fieldsplit/Schur and 3D multi-rank assembly are the next tuning targets.

# Manufactured solutions: why

## Question MMS answers

Does the assembled discrete operator converge to the PDE at the expected order?

- Unit tests check one function; regression checks one accepted result.
- MMS checks the full operator, including faces, ghosts, BCs, geometry and partition boundaries.
- It catches sign/coefficient errors, missed faces and halo bugs that are invisible on one rank.

Salari & Knupp [12]; Roache [11]; Oberkampf & Roy [8].

# Manufactured solutions: method

Choose a smooth $\phi$, substitute into the PDE, and use the residual as the source:

$$s = \mathcal{L}(\phi).$$

For the periodic Laplacian:

$$\phi = \sin(2\pi x)\sin(2\pi y)\sin(2\pi z), \qquad s = -3(2\pi)^2\phi.$$

Refine $h$ and compute

$$p \approx \frac{\log(e_{\text{coarse}}/e_{\text{fine}})}{\log 2}.$$

## Expected orders

Central Laplacian/interpolation: 2. Upwind: 1. TVD on a smooth non-extremum: about 2, with local reduction at extrema.

## Measured on 1, 2 and 4 ranks

Laplacian: 3D 1.978/1.994, 2D 1.994/1.999. Gradient/divergence: 1.967/1.992. Upwind: 0.960/0.990. MUSCL: 1.61/1.57 (minmod), 1.58/1.52 (van Leer), 1.15/1.42 (superbee). The $32^3$ reference $L^2 = 0.1343558076$ is stored.

# Validation roadmap

| Solver | First validation | Reference |
|---|---|---|
| Burgers | viscous traveling wave | exact Cole–Hopf solution |
| Burgers | inviscid shock | characteristics/conservation |
| Heat | 1D boundary layer | exact solution |
| Heat | 2D/3D MMS | manufactured order |
| Stokes | Poiseuille/Couette 2D/3D | exact parabolic/linear |
| Stokes | MMS 2D/3D | manufactured order |
| Stokes | exit boundary | Sankhla & Das (2025) |
| Incompressible | cavity $Re = 100, 1000$ | Ghia et al. [6] |
| Incompressible | Taylor–Green | energy decay/order |

**Rule**

MMS and analytic tests before a physical benchmark. Observations are recorded, including failures and order reduction.

# 1D scalar Burgers: `saturnBurger`

$$\frac{\partial u}{\partial t} + \frac{\partial}{\partial x}\left(\frac{u^2}{2}\right) = \nu \frac{\partial^2 u}{\partial x^2} + s$$

- Rusanov/local Lax–Friedrichs flux with canonical face orientation.
- MUSCL/TVD reconstruction; minmod, van Leer, superbee.
- Shared non-orthogonal Laplacian for diffusion.
- Explicit SSPRK3; Dirichlet and periodic boundaries; time-dependent exact boundary data.
- Registered in the CLI: `saturnCFD case.yaml`, `-listSolvers`, `-dumpConfig saturnBurger`.
- Tests: traveling wave, constant field, periodic inviscid shock.

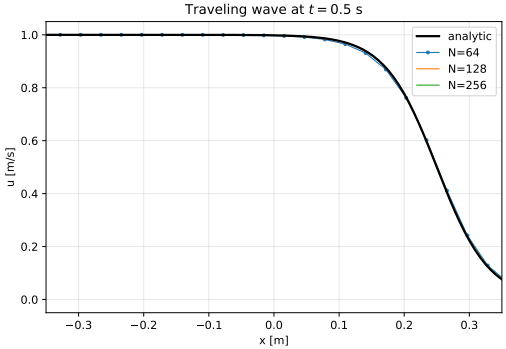
## Measured

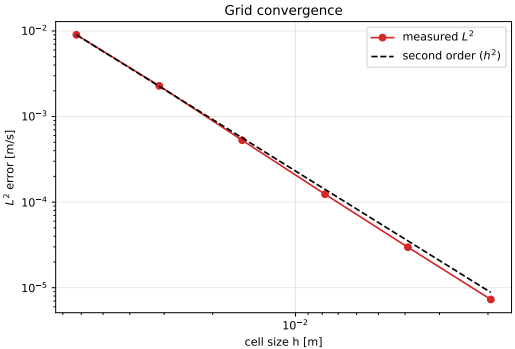
Traveling wave, $N = 32 \to 1024$, $\nu = 0.02$: L2 $9.0 \times 10^{-3} \to 7.3 \times 10^{-6}$, orders 1.98/2.11/2.10/2.06/2.03. At $N = 256$, 1/2/4 ranks give identical norms. 2D diagonal wave: L2 $9.9 \times 10^{-3} \to 6.0 \times 10^{-4}$, orders 1.96/2.10. Shock bounds $[-0.9954, 0.9954]$, integral $1.5 \times 10^{-16}$.

## Why first

Exact solution, nonlinear convection, limiters, time integration, boundary ghosts, conservation and MPI

# Burgers validation against the exact traveling wave

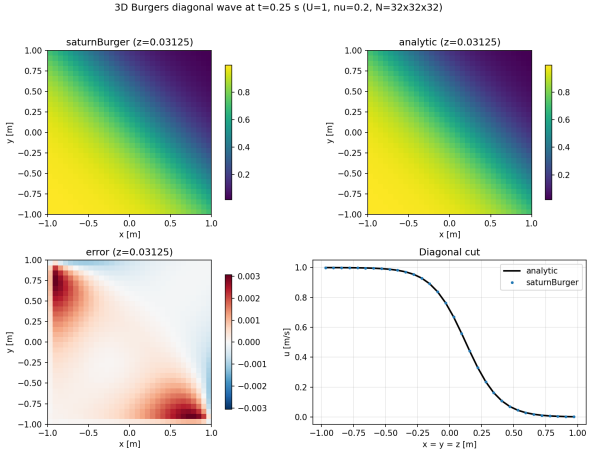



## Verdict

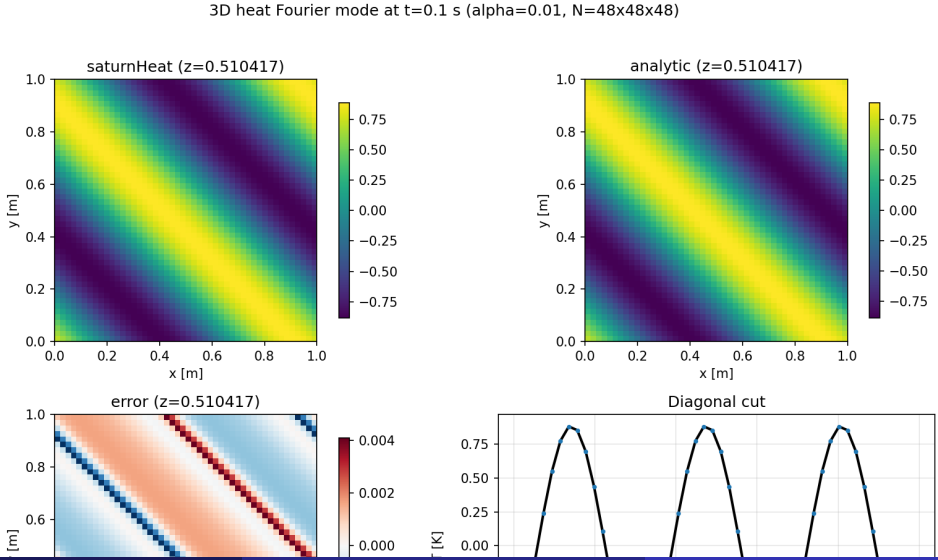
Second-order convergence to the analytic solution, a bounded and conservative shock, and rank-independent norms. The residual error at a finite grid is the expected truncation error, not a solver defect.

# 3D Burgers validation and restart




- 3D conservative flux $F(u)=(u^2/2,u^2/2,u^2/2)$.
- Exact diagonal plane wave.
- $32^3$ example: $L^2 = 8.97 \times 10^{-4}$.
- Coarse 3D plane-wave order 1.37; dense cluster run next.
- `output.checkpointEvery`: binary restart files.
- `restart.file`: resumes at the checkpoint step/time.

# 3D heat validation: periodic Fourier mode



# Build and run

```
# CPU + GPU into one install root
tools/compileSaturnCFD both
#  ~/.local/saturnCFD/{cpu,gpu}
#  ~/.local/bin/saturnCPU, saturnGPU

# run from any directory
saturnCPU /path/to/case.yaml
saturnGPU /path/to/case.yaml
saturnCFD /path/to/case.yaml   # CPU default
```

```
# threads
saturnCPU -threads 1 case.yaml
mpirun -np 4 saturnCPU \
  -threads 6 case.yaml

# GPUs: one rank per GPU
mpirun -np 1 saturnGPU case.yaml
mpirun -np 2 saturnGPU case.yaml

# tests
ctest --preset solver
ctest --preset saturnCFD
ctest --preset gpu
```

The installed binaries carry their backend. `KOKKOS_ROOT` is only needed to configure/build the GPU preset. Test binaries: `testSaturn`, `testSaturnMPI`, `testSaturnFV`, `testSaturnPETSc`.

# Field output: VTK every N steps

```
output:
  prefix: results/burgers/run
  csv: true
  json: true
  vtkEvery: 10   # 0 = off
```

```
results/burgers/
  run_000000.vtu
  run_000010.vtu
  ...
  run_<final>.vtu
  run.pvd        # ParaView
```

## What is written

The scalar cell field `u` at step 0, every `N` accepted steps and the final state. Open the `.pvd` collection in ParaView to animate the sequence. The VTK writer is shared with the mesh `-writeVTK` path.

# Host requirements

```
# pre-flight check
python3 tools/checkRequirements.py

# install missing apt packages
python3 tools/checkRequirements.py \
  --install

# same check through CMake
cmake --build buildCPU --target doctor
```

- `requirements.toml` lists core, docs, editor and GPU tools with minimum versions.
- `checkRequirements.py --install`: PETSc, CUDA toolkit on NVIDIA hosts, and missing Kokkos builds.
- yaml-cpp 0.8 and doctest 2.4.11 are fetched by CMake.
- Missing optional tools are reported, not fatal.

## Uniform names

`saturnCore`, `saturnLinAlg`, `saturnSimulation`, `saturnBurger`; `testSaturn`, `testSaturnMPI`, `testSaturnFV`, `testSaturnPETSc`. Doxygen HTML shows its build date (`TIMESTAMP = YES`).

# Help, clean and completion

```
saturnCFD -help
saturnCFD -help build
saturnCFD -help gpu
saturnCFD -help mpi
saturnCFD -help petsc
saturnCFD -help clangd
saturnCFD -help test
saturnCFD -help clean

source <(saturnCFD -completion bash)

cmake --build buildCPU --target compileDb
clangd --compile-commands-dir=buildCPU

cmake --build buildCPU --target clean
rm -rf buildCPU buildGPU
```

## No memorisation

Test presets: `default`, `cpu`, `gpu`, `solver`, `saturnCFD`. CLI: `saturnCFD -help`. Build: `-help build`. clangd: `-help clangd` and `compileDb`. Clean: `-help clean`.

# Current test status

| Suite | Result |
|---|---|
| CTest total | 46/46 passing |
| MPI distribution & halo | 1, 2, 4 ranks passing |
| Kokkos field + halo | 2 ghosts, 26 directions, 1/2/4 ranks |
| LocalMesh + operators | linear exactness, 1/2/4 ranks |
| Non-orthogonal Laplacian | sheared linear field exact |
| MMS Laplacian | order 2 in 2D/3D, 1/2/4 ranks |
| MMS gradient/divergence | order 1.97/1.99, 1/2/4 ranks |
| MMS upwind/advection–diffusion | order 0.96 and 0.85–0.94 |
| MMS MUSCL | order 1.15–1.61, TVD step bounded |
| Divergence theorem | sheared mesh, roundoff |
| Rhie–Chow | linear zero, checkerboard suppressed |
| PETSc Poisson/nullspace | order 2, 1/2/4 ranks |
| Solver loop | constant, diffusion decay, bounded advection |
| `saturnBurger` | 1D N=32–1024 order 2.03; 2D order 2.10 |
| `saturnHeat` | 3D Fourier order 2.28/2.44; 2D 2.37/2.20 |
| VTK export | 1D frames + PVD; CTest `vtk` |
| Mesh `checkMesh` | pass/reject fixtures passing |
| Config SI-only | accept/reject fixtures passing |

# Status and next steps

## Done

- Mesh + `checkMesh`
- YAML schema + SI validation
- CLI/help/completion/install
- MPI partition + 26-neighbour halo
- rank-0 mesh load + local geometry broadcast
- Kokkos field + device halo
- `LocalMesh` + operators + BC ghosts
- Non-orthogonal Laplacian + upwind/MUSCL convection
- TVD limiters, divergence theorem, Rhie–Chow flux
- Laplacian/gradient/divergence/advection MMS

## Next

- FV coupled matrix assembly
- `saturnStokes`: 2D/3D Poiseuille/Couette validation
- Implicit BE/CN/BDF2
- Incompressible Navier–Stokes
- CUDA build verification
- Cluster scaling study

# Take-away

## What makes this code trustworthy

1. A structured, validated mesh and a strict YAML/SI boundary.
2. An ownership model with no shared cells and a tested halo.
3. A single source for CPU/GPU through Kokkos, not two codes.
4. Verification by manufactured solutions before benchmarks.
5. A CLI and test layout that a user can discover, not memorise.

Questions and discussion

# References I



M. Benzi, G. H. Golub, and J. Liesen.
Numerical solution of saddle point problems.
*Acta Numerica*, 14:1–137, 2005.


C. D. Cantwell, D. Moxey, A. Comerford, A. Bolis, G. Rocco, G. Mengaldo, D. De Grazia, S. Yakovlev, J.-E. Lombard, D. Ekelschot, B. Jordi, H. Xu, Y. Mohamied, C. Eskilsson, B. Nelson, P. Vos, C. Biotto, R. M. Kirby, and S. J. Sherwin.
Nektar++: An open-source spectral/hp element framework.
*Computer Physics Communications*, 192:205–219, 2015.


A. J. Chorin.
Numerical solution of the navier–stokes equations.
*Mathematics of Computation*, 22(104):745–762, 1968.


H. C. Elman, D. J. Silvester, and A. J. Wathen.
*Finite Elements and Fast Iterative Solvers: with Applications in Incompressible Fluid Dynamics*.
Oxford University Press, 2 edition, 2014.

# References II


C. Geuzaine and J.-F. Remacle.
Gmsh: a three-dimensional finite element mesh generator with built-in pre- and post-processing facilities.
*International Journal for Numerical Methods in Engineering*, 79(11):1309–1331, 2009.

U. Ghia, K. N. Ghia, and C. T. Shin.
High-Re solutions for incompressible flow using the navier–stokes equations and a multigrid method.
*Journal of Computational Physics*, 48(3):387–411, 1982.

R. I. Issa.
Solution of the implicitly discretised fluid flow equations by operator-splitting.
*Journal of Computational Physics*, 62(1):40–65, 1986.

W. L. Oberkampf and C. J. Roy.
*Verification and Validation in Scientific Computing*.
Cambridge University Press, 2010.

# References III


S. V. Patankar.
*Numerical Heat Transfer and Fluid Flow*.
Hemisphere Publishing, 1980.

C. M. Rhie and W. L. Chow.
Numerical study of the turbulent flow past an airfoil with trailing edge separation.
*AIAA Journal*, 21(11):1525–1532, 1983.

P. J. Roache.
Code verification by the method of manufactured solutions.
*Journal of Fluids Engineering*, 124(1):4–10, 2002.

K. Salari and P. Knupp.
Code verification by the method of manufactured solutions.
Technical Report SAND2000-1444, Sandia National Laboratories, 2000.

# References IV



K. Sankhla and S. K. Das.
Solution of three-dimensional steady state Stokes flow problems with exit boundary conditions by the boundary element method.
*Computers & Mathematics with Applications*, 195:1–13, 2025.


H. G. Weller, G. Tabor, H. Jasak, and C. Fureby.
A tensorial approach to computational continuum mechanics using object-oriented techniques.
*Computers in Physics*, 12(6):620–631, 1998.